Decision Tree (I)

優點：可解釋性強、效率高

缺點：過擬合、不穩定性（小變動結果不同）、全域最佳解、資料不平衡導致的偏誤

決策樹建構

1. 建立一個根節點，並指派所有訓練資料至其中。
2. 根據特定條件，選擇最佳的分枝屬性。　　分枝準則（Splitting criteria）
3. 在根節點為每項分枝值加入分支。
4. 沿著特定分枝路線，將資料分枝至互斥的子集。
5. 為每個葉節點重複步驟2與3，直到達到停止條件為止。　　停止條件 (Stopping) $\text{Gain} = 0$
6. 修剪 (pruning)

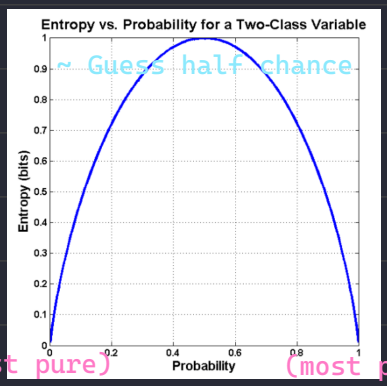
ID3 決策樹演算法 (C4.5)

Entropy (資訊熵)：即不確定性，其值越高則越不確定 e.g. Entropy=1 即半對半錯

$Entropy(S) = -p \cdot \log_2(p) - q \cdot \log_2(q)$ for sample negative p and positive q

Entropy ranges between 0 to 1

$Entropy(S) = \sum_{n=0}^{m} p_n \cdot log_2 \frac{1}{p_n}$ for m classes



(most pure)　　(most pure)

Information Gain (訊息增益)

越大即表示資訊越有意義，如果 Entropy = 1 則該節點為葉節點

$Gain(S, A) = Entropy(S) - \sum_{v \in D_A} \frac{|S_v|}{S} Entropy(S_v)$

Decision Tree (II)

會考 Gini Index ，因為比 Entropy 的 log 好算

Classification and Regression Trees (CART)

Predictor variables: 因子

Homogeneous：同質性

Key Ideas

1. Recursive partitioning

   Repeatedly split the records into two parts so as to achieve maximum homogeneity within the new parts (同質性)

   (a) 遇到數值，就取相鄰兩兩的平均值切分

   (b) 遇到 enum{A, B, C}，就分成 A & (B || C), B & (A || C), C & (A || B)

   (c) Measuring Impurity - Gini Index (CART)

   $$GI(A) = 1 - \sum_{k=1}^{m} (p_k)^2$$

   p_k : proportion of cases in rectangle A that belong to class k

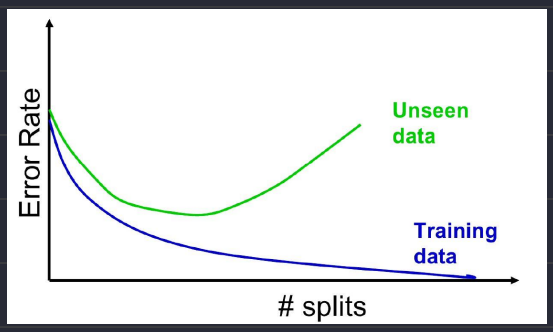
   $GI(A) = 0$: when all cases belong to same class

   $GI(A) = Max$: when all class are equally represented (0.5 in binary case)

   (d) Obtain overall impurity measure (weighted avg)

2. Stop condition

   Natural end of process is 100% purity in each leaf



3. Pruning the tree

   Simplify the tree by pruning peripheral branches to avoid overfitting

Regression Trees

Construction Steps

1. Choose the split points, usually the average value of each nearby points
2. Choose the split point with minimum Impurity $Impurity = RSS = SSE$

   RSS: Residual Sum of Squares

   SSE: Sum Square of differences bewteen actual val and predict val
3. By doing the step 1 and 2 repeatly, until the Impurity = 0
4. Pruning tree

   If impurity of each leaf node is 0, tree would over fit,

   so we need Tree Score to evaluate the best tree size

   $$\text{Tree Score} = R_\alpha(T) = R(T) + \alpha|\bar{T}|$$

   $R(T)$ → 預測值與實際值的誤差平方 $SSE = \Sigma_{i=0}^{N}(\hat{y} - y)^2$

   α → 懲罰係數，越大剪支後樹的複雜度越低

   $|\bar{T}|$ → 葉節點的數量

   Tree Score 越低越好!
5. How to find the best ***α***-value?

   a. Construct a max tree T_{max} using both Training data and Testing data

   b. Keep increasing α until pruning leaves give us lower Tree Score.

   → Keep each α value increased says $\alpha_1, \alpha_2, \alpha_3, \alpha_4, ...$

   c. Divide data into testing and training datasets (k-fold cross validation)

   d. Use founded α values to purne each trees (total number: k)

   e. Calclate the average SSR (SSE) value for each forest built with its alpha

   g. Use the alpha value with lowest SSR (SSE) value with the testing data

Random Forest

Introduction

The core idea is Bagging (Boostrap + Aggreation)

Constructing Random Forest

Let L be original learning set composed of p samples

1. Boostrap Sample Set

   Select q samples from L with replacement (May contain repeated samples)

   The typical value of q is p because around 63% of sample would be selected, then rest of others could be used as testing data $L_k = 100(1 - 1/e) \approx 63\%$

2. Bagging

   Feed same input data to k trees (typically 資料量開根號), then vote for result